

QUANT FINANCE TERMINAL ·
v2.046

SYS://CURRICULUM/D038.MD · PHASE 2 ●
LIVE

LECTURE · 量化金融 365

DAY 038 / 365

P2 · PYTHON 量化工具栈

DEEP DIVE · 8 页深度版

缺失值处理

// Missing Data

KEY CONCEPTS · 三大要点

- ▶ NaN
- ▶ fillna
- ▶ interpolate

01 · WHY THIS LESSON · 这节课为什么重要

本节定调

// 看完这一段你会知道:这节课跟你接下来 3 个月的学习节奏关系有多大

真实行情数据天天有缺口:停牌、未上市、数据缺漏,都会留下一个个空格。这一节用真实 A 股加一段模拟停牌,带你把 NaN 的本质、ffill 和 bfill 前后向填充、interpolate 插值、dropna 的取舍讲透,重点是一个能让回测虚假平稳的致命大坑——停牌期千万不能简单前向填充。

学习目标 · LEARNING OBJECTIVES

// 听课前默念一遍,听完之后回头打勾

- 01 理解 NaN 是什么、为什么不能用等号判断缺失,只能用 isna
- 02 掌握 ffill 前向填充、bfill 后向填充、interpolate 插值三种填补法各自的适用场景
- 03 理解 dropna 不能盲目用:一列缺失会连累整行被删
- 04 看清停牌期 ffill 回测的危害:虚假零波动 + 复牌跳空风险被抹平
- 05 会用时间感知插值 method=time 处理不等间隔的日期缺口

02 · CONTEXT · 历史与背景

小赵用 ffill 填了停牌价,回测净值漂亮得不真实

// 不读历史的策略走不远

新手小赵做了个轮动策略,回测净值曲线又平又稳,夏普高得他自己都不敢信。上实盘却连连踩坑。问题出在数据清洗那一步:他图省事,把所有停牌日的价格都用前一天的收盘价前向填充补上了。这一填,停牌期间每天的收益都变成了零,回测就以为这段时间持仓稳如老狗、毫无波动。可现实里停牌的票根本不能交易,复牌那天往往一次性跳空一大截,把停牌期间积累的利空全砸下来。小赵的回测把这个跳空风险完全抹平了,净值自然漂亮。这一节我们就用真实数据加一段模拟停牌,让你亲眼看到 ffill 怎么把一个 -12% 的复牌跳空,粉饰成一段"零波动"的太平日子。

03 · CORE CONCEPTS · 核心概念详解

把这几个概念彻底搞懂

// 听完视频后,确保你能给一个完全不懂的朋友讲清楚每一条

CONCEPT_01

NaN 不是 None

NaN 是浮点数里专门表示"缺失"的特殊值。它有个反直觉的特性:NaN 不等于任何值,连它自己都不等于自己。所以判断缺失永远用 isna,绝不能用等号去比。

CONCEPT_02

ffill 与 bfill

ffill 前向填充,用缺口前最后一个有效值往后补,好比"沿用昨天的价格";bfill 后向填充,用缺口后第一个有效值往前补。行情数据通常只能 ffill,因为你站在当下看不到未来。

03 · CORE CONCEPTS · 续 (2/3)

CONCEPT_03

interpolate 插值

在缺口两端之间画一条线把中间补上。普通插值按位置等分,method=time 按真实天数加权。跨周末、假期这种不等间隔的缺口,用时间插值更合理。

CONCEPT_04

dropna 的连累效应

dropna 默认只要一行里有任何一个缺失,就把整行删掉。多只票里只要一只停牌,那天其他票的数据也跟着被删,代价很大,所以不能盲目用。

03 · CORE CONCEPTS · 续 (3/3)

CONCEPT_05

停牌不该 ffill 回测

停牌期间根本不能交易,如果用 ffill 把价格补平,回测会误以为这段时间零波动还能持仓,复牌那天的跳空风险被完全抹掉,净值虚假平稳。正确做法是停牌日标 NaN、不参与交易。

04 · PYTHON LAB · 真实可跑代码

缺失值五件套 — isna 识别 / ffill·bfill / dropna 慎用 / 停牌不该 ffill 回测 / interpolate 时间插值

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

► **实操原则:** 先跑通(哪怕硬编码),再优化(参数化/函数化),最后才追求精度(模型升级/特征工程)。散户量化最大的坑是没跑通就开始优化。

```
# day_038_missing.py - 缺失值处理:NaN / isna / ffill / bfill / interpolate / 停牌不  
# 该 ffill 回测  
# 用真实 A 股 + 模拟一段停牌, 识别缺口、演示各种填补法和最致命的"停牌期 ffill 回测"坑  
# 数据:baostock 日线(免费、稳定、国内零翻墙)  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import baostock as bs  
  
pd.set_option('display.width', 150)  
  
STOCKS = {'sh.000300': '沪深300', 'sh.600519': '茅台', 'sz.000002': '万科A', 'sh.  
601318': '中国平安'}  
START, END = '2023-01-01', '2024-12-31'  
  
print('==== 0. 数据拉取(baostock 日线)====')  
lg = bs.login()  
if lg.error_code != '0':  
    raise RuntimeError(f'baostock 登录失败:{lg.error_msg}')  
frames = {}  
for code, name in STOCKS.items():  
    rs = bs.query_history_k_data_plus(  
        code, 'date,close', start_date=START, end_date=END,  
        frequency='d', adjustflag='2')  
    rows = []  
    while (rs.error_code == '0') and rs.next():
```

04 · PYTHON LAB · 真实可跑代码 · 续 (2/5)

缺失值五件套 — isna 识别 / ffill·bfill / dropna 慎用 / 停牌不该 ffill 回测 / interpolate 时间插值

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
rows.append(rs.get_row_data())
s = pd.DataFrame(rows, columns=['date', 'close'])
s['date'] = pd.to_datetime(s['date'])
s['close'] = s['close'].astype(float)
frames[name] = s.set_index('date')['close']
bs.logout()

calendar = frames['沪深300'].index
print(f'完整交易日历(沪深300):{len(calendar)} 天 {calendar[0].date()} → {calendar[-1].date()}')

# 铁律 39:dict 显式映射;reindex 到完整日历
prices = pd.DataFrame({name: frames[name].reindex(calendar) for name in STOCKS.values()})

# 教学:真实 2023-2024 这几只票几乎没停牌,这里手动模拟万科A停牌一段(连续 12 个交易日)
# 真实场景里这种缺口由停牌/退市整理/数据缺漏自然产生,处理逻辑完全一样
HALT_NAME = '万科A'
halt_start = prices.index.get_loc(prices.index[prices.index >= '2024-06-03'][0])
halt_idx = prices.index[halt_start:halt_start + 12]
prices.loc[halt_idx, HALT_NAME] = np.nan
print(f'(教学模拟){HALT_NAME} 停牌 {len(halt_idx)} 个交易日:{halt_idx[0].date()} → {halt_idx[-1].date()}')

# ===== 1. 识别缺失:NaN 不等于 None,先数清楚 =====
print('\n==== 1. 识别缺失值 ====')
na_count = prices.isna().sum()
print('各列缺失(停牌/未上市)天数:')
for name, c in na_count.items():
    print(f' {name:8s} 缺 {c:3d} 天 / 共 {len(prices)} 天 ({c/len(prices)*100:.1f}%)')
print(f'NaN 的特性:NaN == NaN 永远是 {np.nan == np.nan} → 判断缺失只能用 isna,不能用
```



```
==')
```

```
# ===== 2. 小例子讲透 ffill / bfill / interpolate =====
```

04 · PYTHON LAB · 真实可跑代码 · 续 (3/5)

缺失值五件套 — isna 识别 / ffill·bfill / dropna 慎用 / 停牌不该 ffill 回测 / interpolate 时间插值

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print('\n==== 2. 三种填补法(最小例子)====')
demo = pd.Series([10.0, np.nan, np.nan, 13.0, np.nan, 15.0],
index=['周一', '周二', '周三', '周四', '周五', '周六'])
print('原始(周二周三周五停牌):')
print(demo.tolist())
print(f'ffill 前向填充(沿用上一个有效值):{demo.ffill().tolist()}')
print(f'bfill 后向填充(用下一个有效值): {demo.bfill().tolist()}')
print(f'interpolate 线性插值(按位置补中间):{demo.interpolate().round(2).tolist()}')

# ===== 3. dropna 不能盲目用 =====
print('\n==== 3. dropna 不能盲目用 ====')
drop_any = prices.dropna()
print(f'原始 {prices.shape[0]} 天 → dropna(任一列缺就删整行)剩 {drop_any.shape[0]} 天'
f'(为了 {HALT_NAME} 停牌那 {na_count[HALT_NAME]} 天,把当天其他三只票的数据也全删了)')
filled = prices.ffill()
print(f'改成 ffill 填充:保留全部 {filled.shape[0]} 天,缺口用停牌前最后价格暂代')

# ===== 4. 停牌期不该 ffill 回测(最致命的坑)=====
print('\n==== 4. 停牌不该 ffill 回测 ====')
halt_days = int(prices[HALT_NAME].isna().sum())
ret_ffill = prices[HALT_NAME].ffill().pct_change()
zero_in_halt = int((ret_ffill.reindex(halt_idx).abs() < 1e-9).sum())
gap_ret = prices[HALT_NAME].ffill().pct_change().reindex([halt_idx[-1] +
pd.Timedelta(days=1)]).dropna()
# 复牌当天(停牌段之后第一个有效日)的跳空
resume_day = prices.index[halt_start + len(halt_idx)]
resume_ret = prices[HALT_NAME].ffill().pct_change().loc[resume_day]
print(f'演示标的:{HALT_NAME},停牌 {halt_days} 天')
print(f'用 ffill 填充再算日收益:停牌的 {halt_days} 天里有 {zero_in_halt} 天日收益恰好为
0(被填成"零波动")')
print(f'复牌当天({resume_day.date()})一次性跳空 {resume_ret*100:+.2f}%,把停牌期间累积的
```

涨跌全挤在一天'))

```
print('危害:ffill 让回测以为停牌期间也能零波动持仓,复牌跳空风险被抹平 → 回测虚假平稳、收益失真')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (4/5)

缺失值五件套 — isna 识别 / ffill·bfill / dropna 慎用 / 停牌不该 ffill 回测 / interpolate 时间插值

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print('正确做法:停牌日标 NaN 不参与交易,复牌当天的跳空收益如实计入')

# ===== 5. interpolate 时间插值 vs 普通插值 =====
print('\n=== 5. 时间感知插值 ===')
gap = pd.Series([100.0, np.nan, np.nan, np.nan, 108.0],
index=pd.to_datetime(['2024-01-01', '2024-01-02', '2024-01-05', '2024-01-08', '2024-01-09']))
lin = gap.interpolate()
tim = gap.interpolate(method='time')
print('带不等间隔日期的缺口插值:')
print(f' 普通 interpolate(按位置等分):{lin.round(2).tolist()}')
print(f' method=time(按真实天数加权): {tim.round(2).tolist()}')
print('结论:时间序列有不等间隔(跨周末/假期)时,用 method=time 更合理')

# ===== 画图 =====
plt.rcParams['axes.unicode_minus'] = False
fig, ax = plt.subplots(figsize=(12, 4))
mask = prices.isna().T.astype(int)
ax.imshow(mask, aspect='auto', cmap='Reds', interpolation='nearest')
ax.set_yticks(range(len(prices.columns))); ax.set_yticklabels(prices.columns)
ax.set_title('缺失值热力图(红=停牌/缺失日)'); ax.set_xlabel('交易日序号')
plt.tight_layout(); plt.savefig('chart_01.png', dpi=110); plt.close()

fig, ax = plt.subplots(figsize=(10, 5))
xi = range(len(demo))
ax.plot(xi, demo.ffill().values, 'o-', label='ffill 前向', color='#bf616a')
ax.plot(xi, demo.bfill().values, 's--', label='bfill 后向', color='#5e81ac')
ax.plot(xi, demo.interpolate().values, '^:', label='interpolate 插值',
color='#a3be8c')
ax.plot(xi, demo.values, 'kx', markersize=12, label='原始有效值')
ax.set_xticks(list(xi)); ax.set_xticklabels(demo.index)
ax.set_title('三种填补法对比(周二周三周五缺失)'); ax.legend()
```

04 · PYTHON LAB · 真实可跑代码 · 续 (5/5)

缺失值五件套 — isna 识别 / ffill·bfill / dropna 慎用 / 停牌不该 ffill 回测 / interpolate 时间插值

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
plt.tight_layout(); plt.savefig('chart_02.png', dpi=110); plt.close()

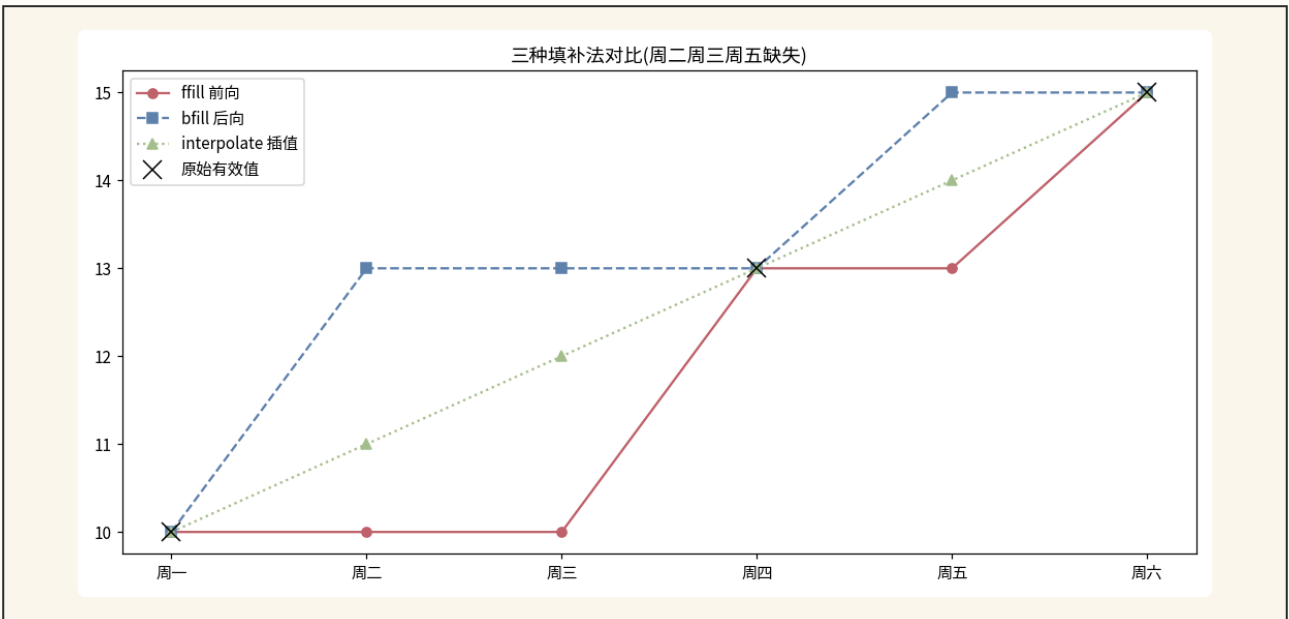
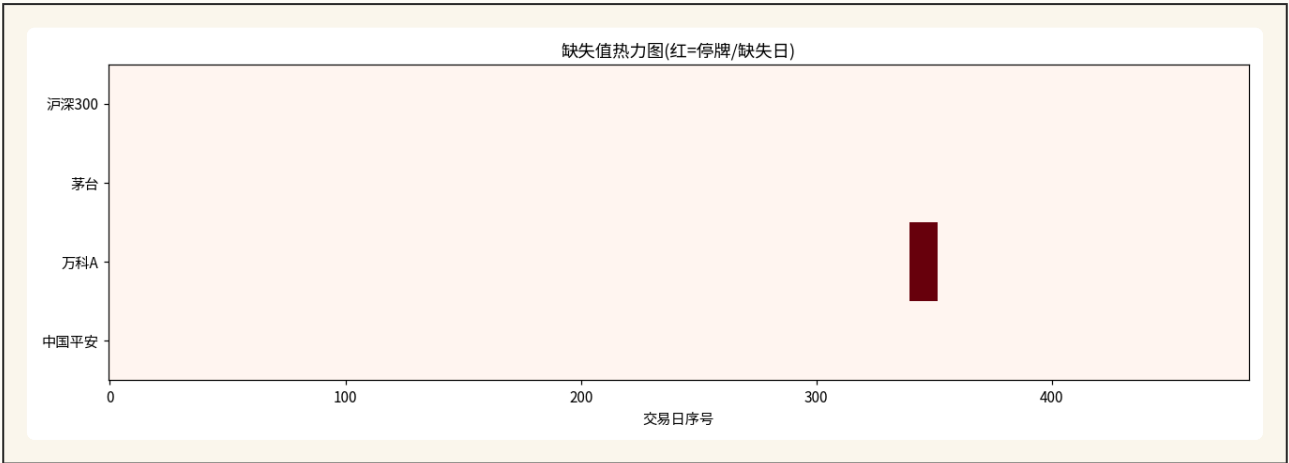
# 图3:停牌段放大 - ffill 价格走平 + 复牌跳空
fig, ax = plt.subplots(figsize=(11, 5))
win = prices.index[max(0, halt_start-10):halt_start+len(halt_idx)+10]
raw = prices[HALT_NAME].reindex(win)
ff = prices[HALT_NAME].ffill().reindex(win)
ax.plot(win, ff, color='#bf616a', lw=1.4, label='ffill 填充(停牌期走平,复牌跳空)')
ax.plot(win, raw, 'o', color='#2e3440', markersize=3, label='真实有效价(停牌期为空)')
ax.axvspan(halt_idx[0], halt_idx[-1], color='#ebcb8b', alpha=0.3, label='停牌期')
ax.set_title(f'{HALT_NAME} 停牌段:ffill 让价格虚假走平,复牌一次跳空'); ax.legend()
plt.tight_layout(); plt.savefig('chart_03.png', dpi=110); plt.close()

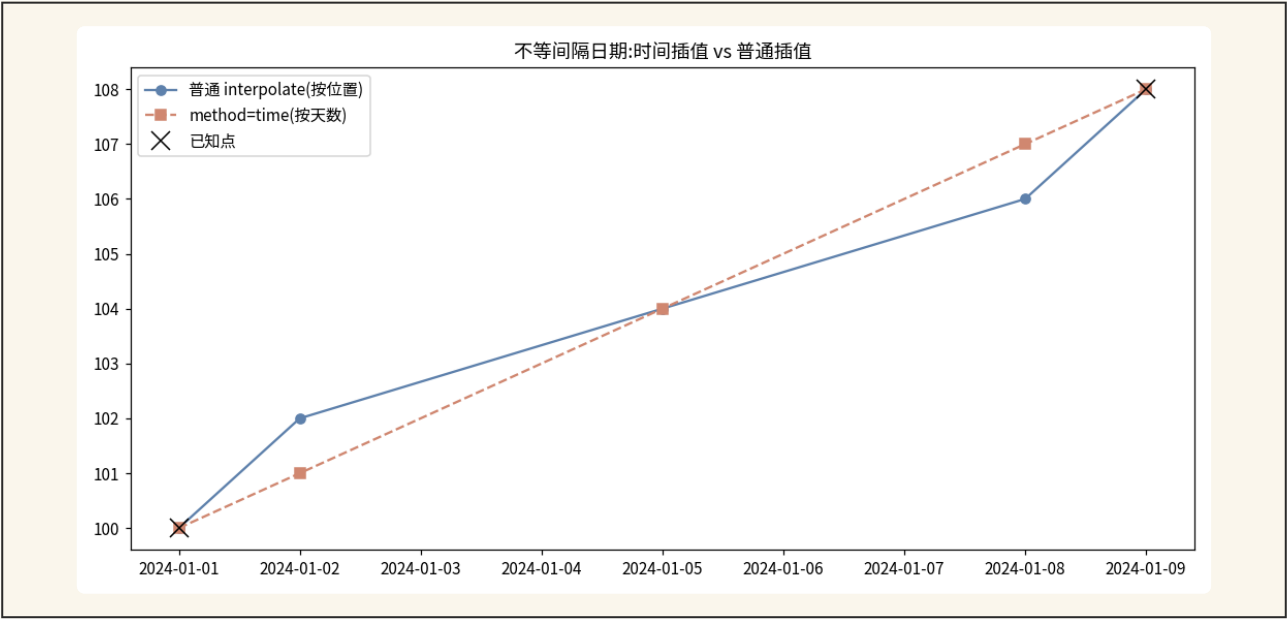
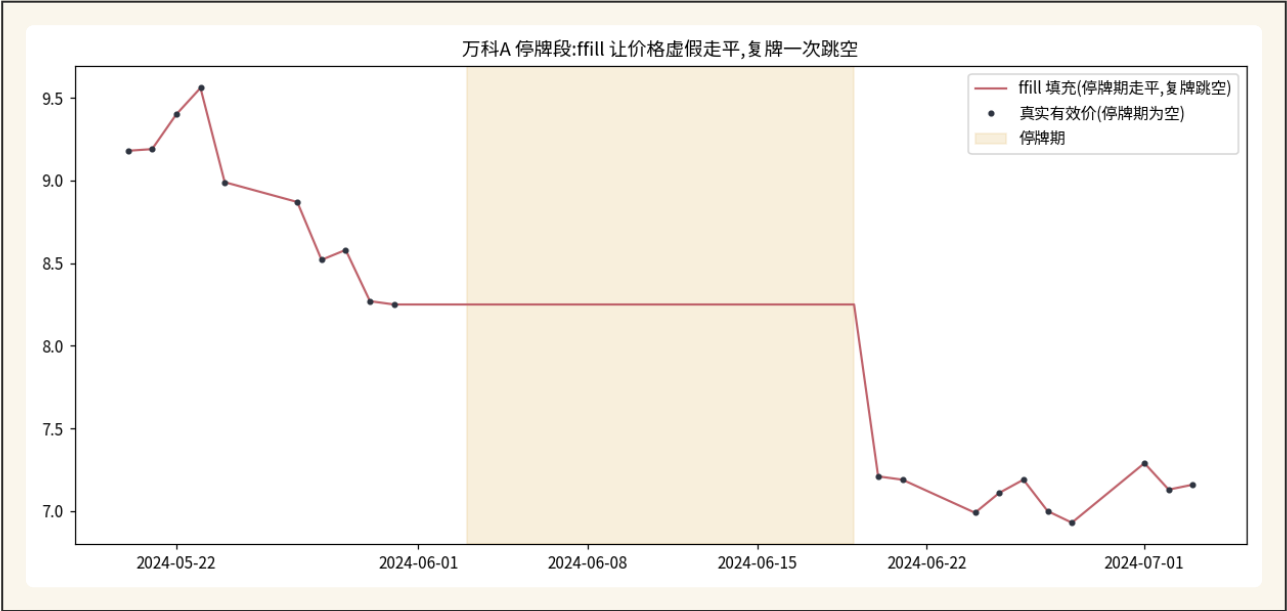
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(gap.index, lin.values, 'o-', label='普通 interpolate(按位置)',
color='#5e81ac')
ax.plot(gap.index, tim.values, 's--', label='method=time(按天数)',
color='#d08770')
ax.plot(gap.index, gap.values, 'kx', markersize=12, label='已知点')
ax.set_title('不等间隔日期:时间插值 vs 普通插值'); ax.legend()
plt.tight_layout(); plt.savefig('chart_04.png', dpi=110); plt.close()

print('\n[done] 4 张图已保存,output.txt 见上方全部打印')
```

回测结果

// · matplotlib 真实输出





NaN 的反直觉特性

NaN == NaN 永远返回 False,连自己都不等于自己;
判断缺失只能用 isna,不能用等号

模拟停牌缺口

把万科A 在 2024-06-03 → 06-19 模拟停牌 12 个交易日(占 484 天的 2.5%),沪深300/茅台/中国平安无缺失

三种填补法对比(小例子
10, NaN, NaN, 13, NaN, 15)

ffill 前向填
[10,10,10,13,13,15];
bfill 后向填
[10,13,13,13,15,15];
interpolate 插值
[10,11,12,13,14,15]

dropna 的连累

整行 dropna 把 484 天削到 472 天:为万科A 停牌 12 天,连带删掉当天其他三只票的有效数据

停牌 ffill 回测的危害

ffill 后停牌 12 天收益全为 0(虚假零波动),复牌当天 2024-06-20 一次性跳空 -12.61%,把停牌期累积的下跌全挤进一天

时间感知插值

不等间隔缺口 [100,_,_,_,108]:普通插值按位置等分 [100,102,104,106,108];
method=time 按真实天数
[100,101,104,107,108]

► 01 **停牌 ffill 把 -12.61% 跳空抹成"零波动":回测最致命的美化**

万科A 模拟停牌 12 天,用 ffill 填平后,这 12 天日收益全是 0,回测以为持仓稳如老狗;可真实里停牌不能交易,复牌当天一次性跳空 -12.61%,把停牌期间积累的利空全砸下来。ffill 把这个跳空风险完全抹平,净值自然虚假平稳——这就是为什么很多人回测夏普漂亮、实盘连连踩坑。停牌日必须标 NaN 不参与交易。

► 02 **NaN 不等于自己:判断缺失只能用 isna**

NaN == NaN 返回 False 是个反直觉但关键的特性,它意味着你永远没法用等号判断一个值是不是缺失,只能用 isna / notna。新手在这里栽跟头,写出的缺失判断永远不成立,数据没洗干净还浑然不觉。

► 03 **dropna 一刀切代价巨大:为 12 天删掉全表**

整行 dropna 只要一行缺就删整行,为了万科A 停牌的 12 天,把这 12 天里沪深300、茅台、中国平安的有效数据也一起陪葬,484 天削成 472 天。多只票拼成的大表,千万别盲目整行 dropna,优先按列处理或只对关注的票处理。

► 04 **ffill 用过去补、bfill 用未来补:行情只能 ffill**

三种填法各有脾气:ffill 沿用上一个有效值(用过去),bfill 用下一个有效值(用未来),interpolate 在两端之间插值。行情数据回测里只能 ffill,因为 bfill 等于用未来的价格补当下,是赤裸裸的未来函数。

► 05 **不等间隔缺口要用时间插值**

同样补 100 到 108 之间的缺口,普通插值按数据点位置等分给出 102/104/106,而 method=time 按真实日期间隔加权给出 101/104/107。跨周末、假期这种不等间隔的时间序列,用时间插值才符合真实的时间流逝。

A 股 · 港股 · 美股 · 加密 全覆盖

// 每个案例都有具体标的和数字,方便你立刻验证

缺失处理	量化回测前清洗数据,停牌日必须标 NaN 排除,而不是 ffill 填平,否则收益和风险全失真
缺失处理	基金净值缺失某日时,用前值还是插值,直接影响最大回撤和波动的计算口径
缺失处理	多只票拼成大表时,新上市的票早期是 NaN,要按上市日各自处理,不能整行 dropna
缺失处理	宏观数据(如月度 CPI)对齐到日频时,常用 ffill 把月值铺到每一天
缺失处理	财务因子有缺失时,行业中位数填补比简单 ffill 更稳健,避免个股极端值

这些坑你不主动避 · 迟早会主动找上你

// 每个坑都附了避坑动作,而不是只描述现象

△ 01

用等号判断缺失

写 `x == NaN` 永远返回假,判断不出缺失。必须用 `isna` 或 `notna`。

△ 02

停牌期 `ffill` 后回测

最致命的坑:停牌日被填平成零波动,复牌跳空风险消失,回测虚假平稳。停牌日应标 `NaN`。

△ 03

盲目 `dropna` 丢大量数据

一列缺就删整行,几只票的零星停牌能让你白白丢掉大量交易日。按列处理或只对关注的票 `dropna`。

△ 04

对未来用 `bfill` 造成未来函数

行情数据 `bfill` 等于用未来的值补当下,回测里就是偷看未来,严禁。

△ 05

不等间隔用普通插值

跨周末假期的缺口用按位置等分的普通插值会失真,应该用 `method=time` 按真实天数加权。

缺失值处理实战 7 条 SOP

// 按这套规则走,你的执行力会立刻拉到中等机构水平

- 01 判断缺失只用 `isna` — 永远不用等号比 `NaN`。
- 02 行情缺口默认 `ffill` — 只能用过去补当下,绝不 `bfill` 偷看未来。
- 03 停牌日标 `NaN` 不交易 — 回测里停牌期不能 `ffill` 成零波动,复牌跳空如实计入。
- 04 `dropna` 前先想清连累 — 多列数据慎用整行 `dropna`,优先按列处理。
- 05 不等间隔用时间插值 — 跨周末假期的缺口用 `method=time`。
- 06 填补前先数清缺口 — `isna().sum()` 看每列缺多少、缺在哪,再决定填法。
- 07 区分缺失原因 — 未上市、停牌、数据缺漏处理逻辑不同,别一刀切。

► **纪律 > 灵感:** 量化做久的人都明白,赚钱的不是某一次绝妙判断,而是把规则坚持执行 1000 次。打印这一页,贴在你电脑边。

把这节课带走的几件事

// 打印或截图这一页, 3 天后再看一遍效果最好

- 01 NaN 是缺失的特殊值,不等于自己,判断只能用 `isna`
- 02 `ffill` 用过去补、`bfill` 用未来补,行情通常只能 `ffill`
- 03 `interpolate` 插值补中间,不等间隔用 `method=time`
- 04 `dropna` 一列缺就删整行,多列数据慎用
- 05 停牌期 `ffill` 会让回测虚假平稳、抹平复牌跳空,必须标 NaN
- 06 行情数据 `bfill` 等于偷看未来,严禁
- 07 填补前先 `isna().sum()` 数清缺口再决定填法

08 · SELF-CHECK · 自测题

答不上来的回看视频对应段落

// 这几道题都没标准答案,目的是逼你自己组织语言讲清楚

Q01 为什么判断缺失不能用等号?(提示:因为 NaN 不等于任何值,连它自己都不等于自己,`x == NaN` 永远返回假。必须用 `isna` 或 `notna` 来判断。)

Q02 为什么停牌期不能用 `ffill` 做回测?(提示:`ffill` 把停牌日价格补平,这几天日收益变成零,回测误以为零波动还能持仓;而真实里停牌不能交易,复牌往往一次性跳空,这个风险被完全抹平,导致回测虚假平稳、收益失真。)

Q03 `ffill` 和 `bfill` 哪个在行情数据里几乎不能用,为什么?(提示:`bfill` 几乎不能用,因为它用缺口后(未来)的值往前补,在回测里等于偷看未来,属于未来函数。)

Q04 普通 `interpolate` 和 `method=time` 的区别?(提示:普通插值按数据点的位置等分填补;`method=time` 按真实日期间隔加权填补。跨周末、假期等不等间隔的缺口,用 `method=time` 更合理。)

09 · NEXT STEP · 下一步

继续往前走

// 看完这节,下面是延伸方向 + 明天预告

推荐阅读 · FURTHER READING

// 听完今天的内容还想往深里挖的话

- ▶ Wes McKinney 《Python for Data Analysis》(2022, O'Reilly)– 缺失值处理章节,ffill/dropna/interpolate 的权威说明
- ▶ pandas 官方文档《Working with missing data》(2024)– NaN 语义与各填补方法的标准参考
- ▶ Marcos López de Prado 《Advances in Financial Machine Learning》(2018, Wiley)– 数据清洗与未来函数(look-ahead bias)的工业级警示
- ▶ Campbell & Lo & MacKinlay 《The Econometrics of Financial Markets》(1997, Princeton)– 停牌、非同步交易对收益估计的影响
- ▶ pandas 官方文档《Time series / date functionality》(2024)– 时间插值 method=time 的用法

NEXT LECTURE · 下一节预告

DAY 039 数据合并 merge/concat

第39课讲数据合并:用 merge 像 SQL 一样按键拼表,用 concat 沿轴拼接,还有量化里最实用的 merge_asof 时间就近对齐——把每天的股价和最近一期已披露的财报安全地对上,杜绝偷看未来。